

TimeSeriesFlow and AdaptiveForecast

Comparative Evaluation and Commercial Impact Report

TimeSeriesFlow Open Benchmark

2026-06-15

Abstract

We benchmark four entity processing approaches on three public multi-entity time series datasets. This Phase 1 report answers a single question: should you adopt TimeSeriesFlow, AdaptiveForecast, or the combined stack for your workload? We measure screening throughput, gate outcomes, routing diversity, and orchestration overhead. We do **not** yet measure forecast accuracy (Phase 2). Vanilla pandas wins on raw speed for simple statistics. AdaptiveForecast blocks unfit entities and assigns model families. TimeSeriesFlow adds batch structure with modest overhead on lightweight work.

1 Executive verdict

Bottom line. Across 40 entities, TimeSeriesFlow is an orchestration layer (~14x pandas on stats-only work), AdaptiveForecast adds screening and routing on Intel Berkeley Lab, ETT Hourly (ETT_{h1}), and forecast accuracy remains unmeasured. UCI Household Power blocked every entity under current gates (screening worked; not a deployment-ready forecast slice). Full-stack screening averages 60 entities/s.

1.1 When to use the stack

- Entity count is large enough that manual loops, retries, and progress matter
- Entity quality varies and some series should be blocked before training
- A single global model family is risky and per-entity routing is required
- You accept profiling overhead in exchange for screening and orchestration

1.2 When not to use the stack

- You only need exploratory mean, count, or simple aggregates on a small fleet
- All entities are homogeneous and pass the same model family
- You must prove forecast accuracy gains (not measured in Phase 1)
- Raw pandas throughput is the primary success metric

1.3 Effectiveness decision table

Notes.

- **Faster than pandas for the same lightweight stats task:** TimeSeriesFlow adds a mean 13.6x slowdown on stats-only work; profiling dominates full-stack cost.

Table 1: Pass or fail against benchmark criteria (Phase 1: no forecast accuracy test).

Criterion	Pandas	TSFlow	AF loop	Full stack
Faster than pandas for the same lightweight stats task	Pass	Partial	Fail	Fail
Blocks unfit entities before model training	N/A	N/A	Pass	Pass
Assigns different model families per entity	N/A	N/A	Pass	Pass
Forecast accuracy better than a single global model	N/A	N/A	Not tested	Not tested
Production batch ops (checkpoints, parallel workers, retries)	Fail	Not tested	N/A	Not tested
Screening throughput for nightly fleet batches	N/A	N/A	Partial	Partial

- **Blocks unfit entities before model training:** 5 of 40 entities blocked portfolio-wide. All entities blocked on: UCI Household Power.
- **Assigns different model families per entity:** Mean routing diversity 7.0%; up to 2 unique families assigned.
- **Forecast accuracy better than a single global model:** Phase 2 backtest not run. No MAE or sMAPE in this report.
- **Production batch ops (checkpoints, parallel workers, retries):** Benchmark used workers=1 and did not exercise checkpoint resume.
- **Screening throughput for nightly fleet batches:** Mean full-stack screening: 59.6 entities/s.

1.4 TimeSeriesFlow

Verdict (Pass): Effective as an orchestration layer (14x slower than pandas on the same lightweight stats task; acceptable overhead for batch structure).

Use when:

- Structured per-entity batch runs with retries and progress hooks
- Teams that outgrow ad-hoc groupby loops but do not need profiling yet

Do not use when:

- Beating pandas on simple mean and row-count aggregates
- Proving forecast quality (orchestration only in this benchmark)

1.5 AdaptiveForecast

Verdict (Partial): Partially effective: screening blocks unfit entities before training; routing assigns multiple model families where patterns differ; on UCI Household Power all entities failed gates (short span slice; screening-only outcome). Forecast accuracy was not tested.

Use when:

- Screening entities that fail quality gates before training spend
- Routing labels when entity patterns differ within a fleet

Do not use when:

- Replacing pandas for raw speed on simple statistics
- Claims of better forecasts without a Phase 2 accuracy backtest

1.6 Full stack

Verdict (Partial): Partially effective when combines screening with batch orchestration and delivers per-entity routing in one pass at ~60 entities/s (+0.02s mean vs AdaptiveForecast-only loop). Not faster than pandas; value is operational plus screening, not speed.

Use when:

- Nightly screening plus orchestration on heterogeneous entity fleets
- Workflows that need both gate outcomes and batch runner structure

Do not use when:

- Small homogeneous fleets where a single model and a pandas loop suffice
- Latency-sensitive paths that only need lightweight aggregates

2 Introduction

Large fleets of heterogeneous time series create two recurring costs: wasted training on entities that fail quality gates, and forecast error from a single global model family. TimeSeriesFlow addresses batch orchestration; AdaptiveForecast adds profiling, validation gates, and per-entity model routing. This report measures when the combined stack justifies its runtime overhead relative to manual pandas workflows.

3 Methodology

3.1 Datasets

Table 2: Open datasets used in the benchmark.

Dataset	Domain	Entities	Max rows/entity
Intel Berkeley Lab	Wireless sensors	30 (subset)	4 000
ETT Hourly (ETTh1)	Grid load	7	4 000
UCI Household Power	Residential meters	3	4 000

3.2 Approaches

- **Vanilla pandas:** groupby loop computing mean and row count only.
- **AdaptiveForecast loop:** per-entity profiling and routing without TimeSeriesFlow.
- **TimeSeriesFlow:** EntityRunner with lightweight per-entity summaries.
- **Full stack:** EntityRunner plus AdaptiveForecast profiling, validation gates, and routing.

All timed runs use a single worker unless noted. Metrics are computed on identical entity subsets per dataset.

4 Commercial key performance indicators

Table 3 summarizes pitch-ready metrics derived from full-stack runs. Portfolio gate block rate is 12.5% across 40 entities. Mean screening throughput is 59.6 entities/s.

Table 3: Operational KPIs per dataset (full stack). Training jobs avoided equals gate block rate. Profiling cost is vanilla throughput divided by screening throughput.

Dataset	Entities	Jobs avoided	Diversity	Models	Screen (ent/s)	Cost factor
Intel Berkeley Lab	30	6.7%	6.7%	2	45.4	98.8×
ETT Hourly (ETTh1)	7	0.0%	14.3%	2	72.8	38.9×
UCI Household Power	3	100.0%	0.0%	1	60.6	32.0×

Metric definitions.

- **Training jobs avoided:** share of entities blocked by validation gates before model assignment.
- **Routing diversity:** share of entities assigned a model different from the fleet modal choice.
- **Screening throughput:** entities profiled and routed per second under the full stack.
- **Profiling cost factor:** how many times slower full-stack screening is versus vanilla stats-only loops.

4.1 Executive highlights

- Across 3 datasets, 12.5% of entities would be blocked before model training.
- Mean routing diversity is 7.0% with up to 2 unique architectures assigned.
- Full-stack screening averages 59.6 entities per second.
- UCI Household Power: 100% training waste avoided (3 of 3 entities blocked).

5 Results

6 Per-dataset benchmark tables

6.1 Intel Berkeley Lab

Classic wireless sensor network with 54 motes and irregular sampling intervals.

Approach	Seconds	Entities/s	Gate pass	Routing diversity
vanilla_pandas	0.01	4480.31	—	—
adaptiveforecast_loop	0.56	53.80	93.3%	6.7%
timeseriesflow	0.16	188.98	—	—
full_stack	0.66	45.37	93.3%	6.7%

Entities: 30. Rows: 114372. Median rows/entity: 4000. Span (days): 4.8.

6.2 ETT Hourly (ETTh1)

Seven electricity transformer load channels on an hourly grid.

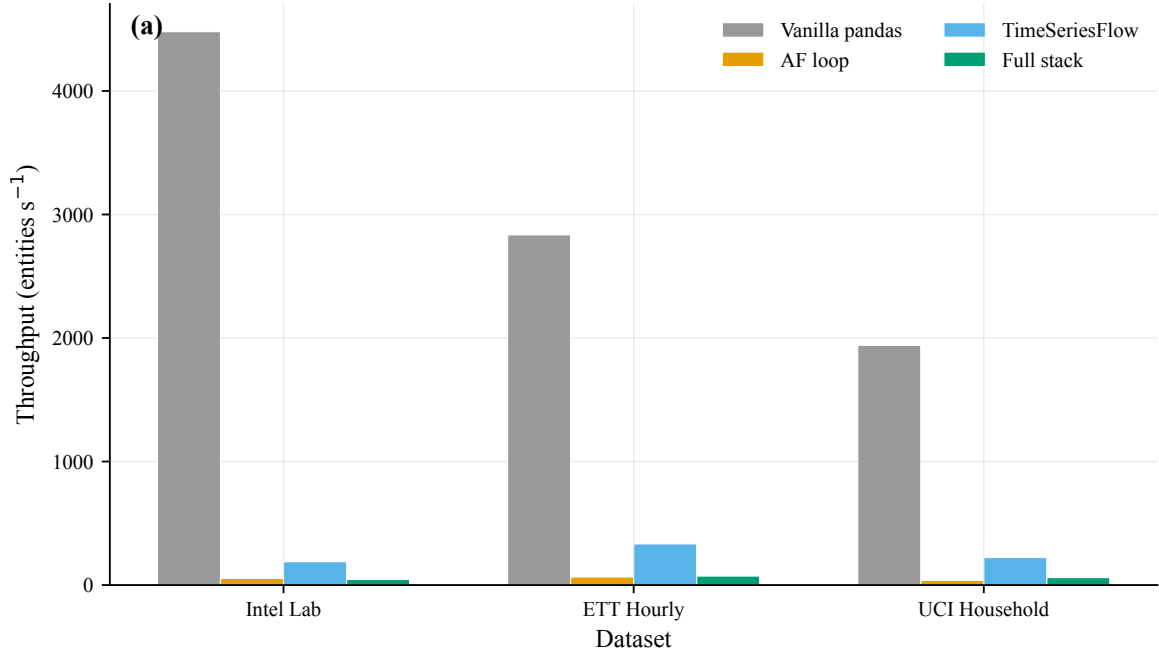
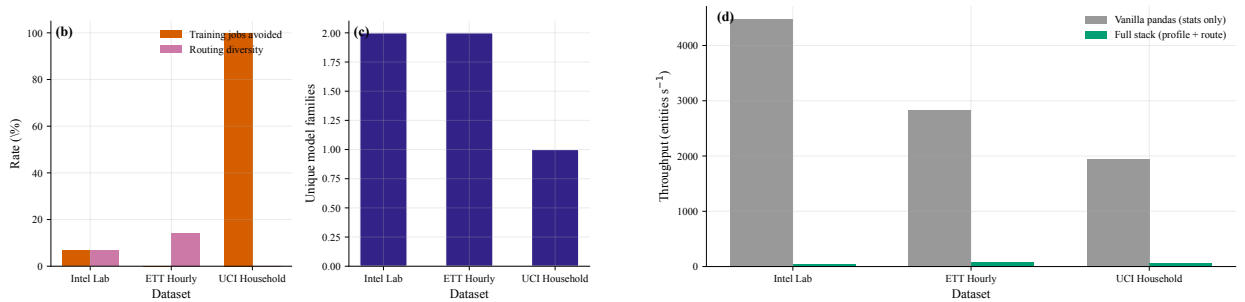


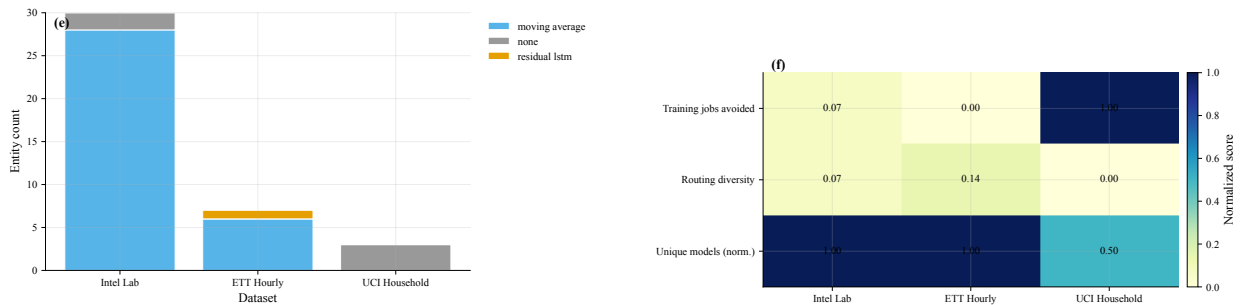
Figure 1: Entity throughput across datasets and processing approaches. Vanilla pandas maximizes raw speed; the full stack trades throughput for profiling and routing.



(a) Training jobs avoided, routing diversity, and unique model families.

(b) Vanilla stats throughput versus full-stack screening throughput.

Figure 2: Commercial impact and throughput trade-off.



(a) Per-dataset model family assignment counts.

(b) Normalized KPI heatmap across datasets.

Figure 3: Routing outcomes and cross-dataset KPI comparison.

Approach	Seconds	Entities/s	Gate pass	Routing diversity
vanilla_pandas	0.00	2835.20	—	—
adaptiveforecast_loop	0.11	65.59	100.0%	14.3%
timeseriesflow	0.02	333.33	—	—
full_stack	0.10	72.85	100.0%	14.3%

Entities: 7. Rows: 28000. Median rows/entity: 4000. Span (days): 166.6.

6.3 UCI Household Power

Three residential sub-meter channels with minute-level power readings.

Approach	Seconds	Entities/s	Gate pass	Routing diversity
vanilla_pandas	0.00	1940.44	—	—
adaptiveforecast_loop	0.08	36.99	0.0%	0.0%
timeseriesflow	0.01	223.07	—	—
full_stack	0.05	60.64	0.0%	0.0%

Entities: 3. Rows: 12000. Median rows/entity: 4000. Span (days): 2.8.

7 Discussion

The executive verdict above is the primary takeaway. The sections below provide supporting metrics.

How to read throughput charts. Do not treat vanilla pandas throughput as a forecast baseline. It computes mean and row count only. Full-stack throughput reflects profiling plus routing plus orchestration. Compare TimeSeriesFlow to pandas for orchestration tax; compare full stack to AdaptiveForecast-only for runner overhead.

Commercial framing. Lead with screening and routing when entity quality varies. Lead with orchestration when batch reliability matters. Do not sell on pandas speed. Phase 2 must add forecast error metrics before claiming model-selection effectiveness.

8 Conclusions

- TimeSeriesFlow: Effective as an orchestration layer (14x slower than pandas on the same lightweight stats task; acceptable overhead for batch structure).
- AdaptiveForecast: Partially effective: screening blocks unfit entities before training; routing assigns multiple model families where patterns differ; on UCI Household Power all entities failed gates (short span slice; screening-only outcome). Forecast accuracy was not tested.
- Full stack: Partially effective when combines screening with batch orchestration and delivers per-entity routing in one pass at ~60 entities/s (+0.02s mean vs AdaptiveForecast-only loop). Not faster than pandas; value is operational plus screening, not speed.
- Across 40 entities, TimeSeriesFlow is an orchestration layer (~14x pandas on stats-only work), AdaptiveForecast adds screening and routing on Intel Berkeley Lab, ETT Hourly (ETTth1), and forecast accuracy remains unmeasured. UCI Household Power blocked every entity under current gates (screening worked; not a deployment-ready forecast slice). Full-stack screening averages 60 entities/s.

Report generated by TimeSeriesFlow open benchmark tooling. Figures exported at 300 DPI with vector PDF companions.